

RELATÓRIO DE EVOLUÇÃO DO PROJETO

1. RESUMO DA EVOLUÇÃO

Sprint 1 (planejamento): Definição da persona (motorista de EV), do escopo do chatbot, das tecnologias propostas (OpenAI GPT-4o + FastAPI + OCPP), fluxograma de funcionamento, system prompt inicial em texto corrido e modelo de teste com 6 perguntas/respostas ideais.

Sprint 2 (implementação inicial): Núcleo conversacional funcional em Python puro (Google Colab), usando a API do Google Gemini (gemini-2.5-flash) chamada diretamente via SDK, com histórico mantido em uma lista Python simples (sem limite de tokens, sem validação de saída, sem guardrails automatizados) e dados de sessão fixos injetados como string no prompt.

Sprint 03: Refactory completo do núcleo conversacional para LangChain (LCEL), trocando a chamada direta de SDK por uma chain composável ChatPromptTemplate | ChatOllama | parser, rodando sobre o Ollama Cloud (modelos gpt-oss:120b e qwen3:8b) em vez do Gemini. Foram adicionados: memória por sessão com limite de tokens, saída estruturada validada por schema Pydantic v2 (ConsultaRecarga), system prompt versionado com XML tagging, dois guardrails determinísticos (moderação anti-jailbreak e validação de escopo GoodWe) que rodam antes de qualquer chamada ao modelo, e um eval set reexecutável com 16 casos cobrindo happy path, edge cases, jailbreak e out-of-scope.

Comparativo de Arquitetura (Sprints 1/2 vs. Sprint 03)

Sprints 1/2 (Legado)	Sprint 03 (Refatorado)
Chamada direta ao SDK (Gemini)	Chain LCEL (prompt llm parser) sobre Ollama Cloud
Histórico em lista Python simples (sem limite)	Memória por sessão com poda automática por limite de tokens
Resposta sempre em texto livre	Texto livre OU saída estruturada validada (Pydantic v2)
System prompt em texto corrido	System prompt versionado (v1 → v2) com XML tagging
Nenhum guardrail automatizado	Guardrail de moderação (jailbreak) + guardrail de escopo (jurídico, financeiro, segurança elétrica)
Eval manual (6 perguntas, sem script de reexecução)	Eval set de 16 casos, reexecutável por script (evals/run_evals.py)

2. REFATORAÇÃO — DECISÕES TÉCNICAS E TRADE-OFFS

2.1 Migração de SDK direto (Gemini) para LCEL sobre Ollama Cloud

Optamos por ChatOllama(base_url="https://ollama.com", client_kwargs={"headers":{"Authorization": "Bearer <API_KEY>"}}) em vez de instalar Ollama localmente. *Trade-off:* dependemos de conectividade com a nuvem da Ollama (assim como já dependíamos da nuvem do Gemini na Sprint 2), mas ganhamos acesso a gpt-oss:120b sem custo de GPU própria — o mesmo raciocínio de custo/benefício já usado na Sprint 1 para justificar a OpenAI API em vez de um modelo local.

2.2 Memória: RunnableWithMessageHistory + equivalente ao ConversationTokenBufferMemory

As instruções da Sprint 03 pedem explicitamente ConversationTokenBufferMemory. Essa classe pertence à API "legada" de memória do LangChain (langchain.memory) e foi REMOVIDA nas versões atuais da biblioteca — a

2.3 Saída estruturada via PydanticOutputParser (não .with_structured_output)

Optamos por instruir o formato de saída via prompt (format_instructions do PydanticOutputParser) em vez de depender de function calling nativo do provedor. *Trade-off*: um pouco mais de tokens no prompt, mas compatibilidade garantida com qualquer modelo do Ollama Cloud, mesmo os que não expõem tool calling estruturado de forma confiável.

2.4 Guardrails determinísticos antes da chain, não dentro do prompt

Em vez de confiar apenas em instruções de sistema para o modelo recusar jailbreak/escopo indevido (abordagem sujeita a falhas do próprio modelo), implementamos dois guardrails em código Python puro (src/guardrails/moderation.py e scope_validator.py) que rodam ANTES da chamada ao LLM. *Trade-off*: heurística por regex pode gerar falsos positivos/negativos em casos muito criativos de jailbreak, mas garante custo zero de API para bloquear os casos óbvios e uma segunda camada de defesa (o system prompt v2 também instrui o modelo a recusar, caso a heurística não capture algum caso).

■ 3. TABELA DE COMPARATIVO ANTES/DEPOIS

Comandos para execução dos testes automatizados de avaliação:

```
export OLLAMA_API_KEY="sua_api_key" python evals/run_evals.py --prompt v1 --model gpt-oss:120b (baseline) python evals/run_evals.py --prompt v2 --model gpt-oss:120b (refatorado)
```

Métrica	Sprints 1/2 (Legado)	Sprint 03 (LCEL)
Qualidade das respostas (nota no eval, 16 casos)	Básicas (rodar eval manual sobre sprint_2_paa)	Ótimas (evals/run_evals.py --prompt v2)
Tokens por turno (média)	Sem medição	116.5
Latência média por turno (s)	Sem medição	0.686
Acurácia do structured output	N/A (sem saída estruturada na v2)	10/10 (validação Pydantic passa/falha)

■ 4. PROBLEMAS ENCONTRADOS E SOLUÇÕES

PROBLEMA 1
ConversationTokenBufferMemory não existe mais no LangChain atual
A classe pedida no escopo pertence a uma API descontinuada (langchain.memory), incompatível com a versão do langchain-ollama necessária para falar com o Ollama Cloud (testamos e confirmamos o conflito de dependências ao tentar instalar uma versão antiga do langchain).

Decisão: Reimplementar o mesmo comportamento (memória por sessão com corte automático por limite de tokens) usando BaseChatMessageHistory + trim_messages + tiktoken, mantendo a mesma interface exigida pelo RunnableWithMessageHistory. Resultado testado: em um cenário de 3 turnos com orçamento reduzido de tokens, o turno mais antigo foi automaticamente descartado, preservando os mais recentes — mesmo comportamento esperado do buffer original.

PROBLEMA 2
Verificação automática de qualidade de resposta em linguagem natural
Para os casos de "happy path" e "edge case" do eval set, a resposta do modelo é texto livre em português — não há uma forma trivial de verificar automaticamente, por regex, se a resposta "está correta" sem um modelo juiz adicional (fora do

PROBLEMA 3

Modelo real omitindo campo obrigatório na saída estruturada

Ao testar a chain de saída estruturada (`consulta_estruturada.py`) com o `gpt-oss:120b` real via Ollama Cloud, o modelo devolveu um JSON quase completo, mas sem o campo obrigatório `resumo_para_usuario`, causando `OutputParserException`. O `field_validator/campo obrigatório` do Pydantic funcionou exatamente como deveria (barrou um dado incompleto em vez de deixar passar), mas isso quebrava a experiência em produção.

Decisão: (1) reforçamos o prompt reafirmando explicitamente que todos os campos são obrigatórios e adicionando um exemplo completo de JSON válido (few-shot de 1 exemplo); (2) como a versão atual do LangChain não inclui mais o `OutputFixingParser` pronto (também removido da API legada, mesma causa-raiz do Problema 1), implementamos um wrapper de retry manual (`montar_chain_estruturada_com_retry`, em `src/chain/consulta_estruturada.py`) que reenvia ao modelo o JSON inválido junto com o erro de validação exato, pedindo a correção, por até 2 tentativas. Testado com um cenário simulado (primeira resposta inválida, segunda corrigida) e confirmado funcionando.

5. EQUIPE E DIVISÃO DE TRABALHO

Nome	RM	Tarefa Principal na Sprint 03
Wendel Pedro	573126	• Refactory da chain LCEL (<code>src/chain/builder.py</code>), memória por sessão com limite de tokens e guardrails (moderação + escopo) • System prompt versionado (<code>v1</code> → <code>v2</code>, XML tagging) e tabela de versões (<code>prompts/versoes.md</code>) • Schema Pydantic v2 (<code>ConsultaRecarga</code>) e chain de saída estruturada (<code>src/schemas/</code>, <code>src/chain/consulta_estruturada.py</code>) • Relatório de uso de modelos e parâmetros (<code>docs/relatorio_modelos.md</code>), comparação <code>gpt-oss:120b</code> vs <code>qwen3:8b</code> • Construção e execução real do eval set (<code>evals/eval_set.json</code>, <code>evals/run_evals.py</code>), preenchimento da tabela antes/depois (Seção 3) • Relatório de evolução do projeto (este documento), <code>README.md</code> e organização/commits do repositório
Daniel Alejandro	573075	Outras Sprints
Victor Hugo	573838	Outras Sprints
Arthur Araújo	573308	Outras Sprints

Turma: [1CCPK]

Projeto: FIAP × GoodWe Brasil (EV Challenge 2026)